

Software Development Fundamentals (SDF-1)

15B11CI111

ODD 2023-2024



Introduction to the Course

Course Outline

- **Course Outline**
- **Course Outcome**
- **Evaluation Criteria**
- **Text/Reference Books, e-Resources**

Course Outline/Curriculum & Lecture Count

Module No.	Subtitle of the Module	Topics in the Module	No. of Lectures for the module
1.	Introduction	Introduction to Software Development Life Cycle, Step by step solution to simple problems, developing logic/flow- chart/pseudo code to solve problems like 2D screen saver, simple/logical games, puzzles	6
2.	Data types, operators, and Control Flow	Data, variables and constants, data types, operators – binary, unary, ternary, operator precedence, operations using different operators, if, if-else, while, do-while, for, switch-case in C Programming	8
3.	Array	Fundamentals of Array, Implementation of 1D/2D Array and related operations like insertion, traversal, updation, etc. in C programming using different problems	7
4.	Pointers	Pointers in C, Dynamic memory allocation for 1D/2D array, Arithmetical operations on pointers	5
5.	Functions	Introduction to Functions and its implementation in C programming language, Functions using Pass by value, functions using pass by reference, recursive functions	5
6.	Structures and Union	Introduction and implementation of Structures and Union in C programming, Array of Structures, Pointer to Structures and related operations like insertion, traversal, updation, etc. in C programming using different problems, Structures using function	5
7.	File Handling	Introduction to File, creation of files in C programming language, Modes of File Handling like read, write, update; different types of files like binary file and text file and respective operations like, opening, closing, reading, writing, end of file, traversing the file, for structured and unstructured data	6
Total number of Lectures			42

Course Outline/Curriculum & Lecture Count

- **Project Based learning:** In this subject, students work in the team of 3-4 people, to implement a small application/mini-project based on the learned concepts. The students will be able apply various concepts of SDLC lifecycle, C pointers, functions, arrays, structures, union and file handling for developing a real life application. This will aid in their employability in software industry.

Course Outcomes (COs)

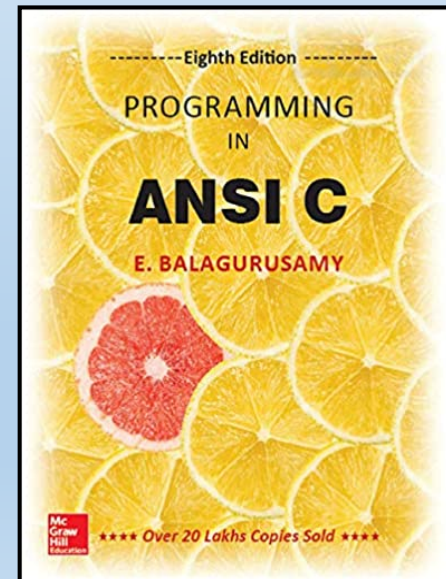
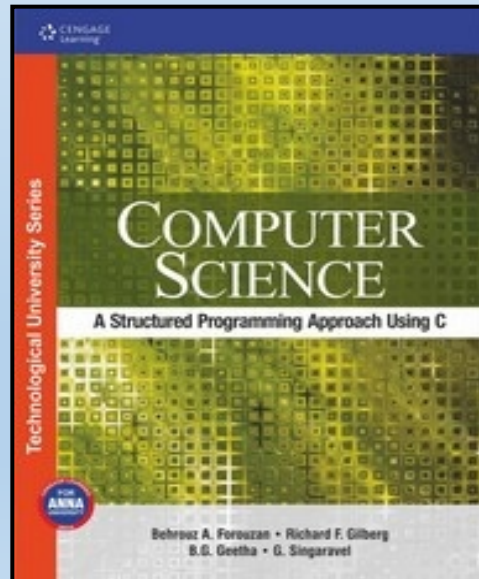
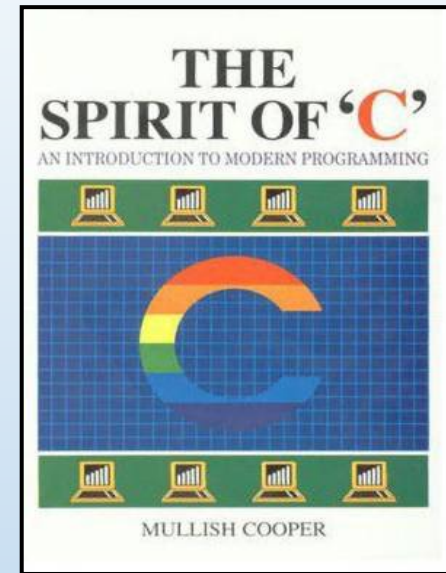
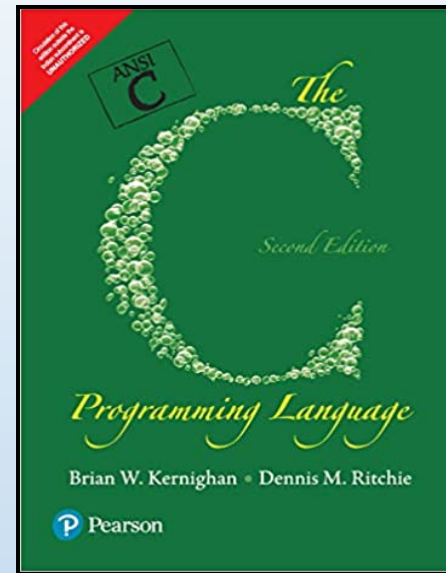
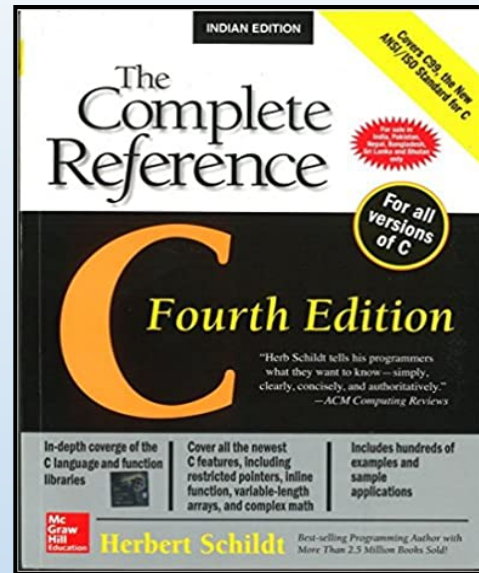
Course Outcomes (CO)		Cognitive Levels
CO1	Explain various phases of software development life cycle	Understand Level (Level 2)
CO2	Explain various data types, memory allocation schemes, precedence of arithmetical and logical operations, and need of array, and structures	Understand Level (Level 2)
CO3	Design the flow chart and write the high level code for different problems	Understand Level (Level 2)
CO4	Apply and implement functions with or without pointers for different problems	Apply Level (Level 3)
CO5	Demonstrate and implement various operations like traverse, insertion, deletion, <i>etc.</i> on files	Apply Level (Level 3)

SDF – I: Evaluation Criteria

S.No.	Evaluative Components and tentative dates of examination	Marks
1	T1 Examination (Tentative date: Sept 5 – Sept 12, 2023)	20
2	T2 Examination (Tentative date: Oct 16 – Oct 21, 2023)	20
3	End Semester Examination (Tentative date: Dec 04 – Dec 12, 2023)	35
4	Teachers Assessment (Attendance = 10, Class Test, Quizzes, etc = 05, Internal assessment = 05, Assignments in PBL mode = 05)	25
	Total	100

Text/Reference Books

S. No.	Text/Reference Books
1	Herbert Schildt. "The Complete Reference C ", 4th Edition, TMH, 2017
2	Brian W. Kernighan and Dennis M. Ritchie , "The C Programming Language", 2nd Edition, Pearson Education India, 2015
3	H. Cooper and H. Mullish, Jaico Publishing House. "The Spirit of C", 4th Edition, Jaico Publishing House, 2006
4	B. A. Forouzan, R. F. Gilberg, B. G. Geetha, and G. Singaravel "Computer Science: A Structured Programming Approach Using C", 1st Edition, Cengage, New Delhi, 2012
5	Ashok N. Kamthane and Amit A Kamthane, "Programming in C", 3 rd Edition, Pearson Education, 2015
6	E. Balagurusamy, "Programming in ANSI C", Eighth edition, TMH, 2019



e-Resources (Courses (Theory/Practical) on Computer Programming using C)



About usCoursesCont

CoursesComputer Science and EngineeringNOC:Introduction to programming in C (Video)**Syllabus**Co-ordinated by : IIT Kanpur

Available from : 2014-09-25Lec :1

Modules / Lectures

Introduction to programming in C

Live Session

Watch on YouTubeVideoAssignmentsDownload Videos**Transcripts**Books

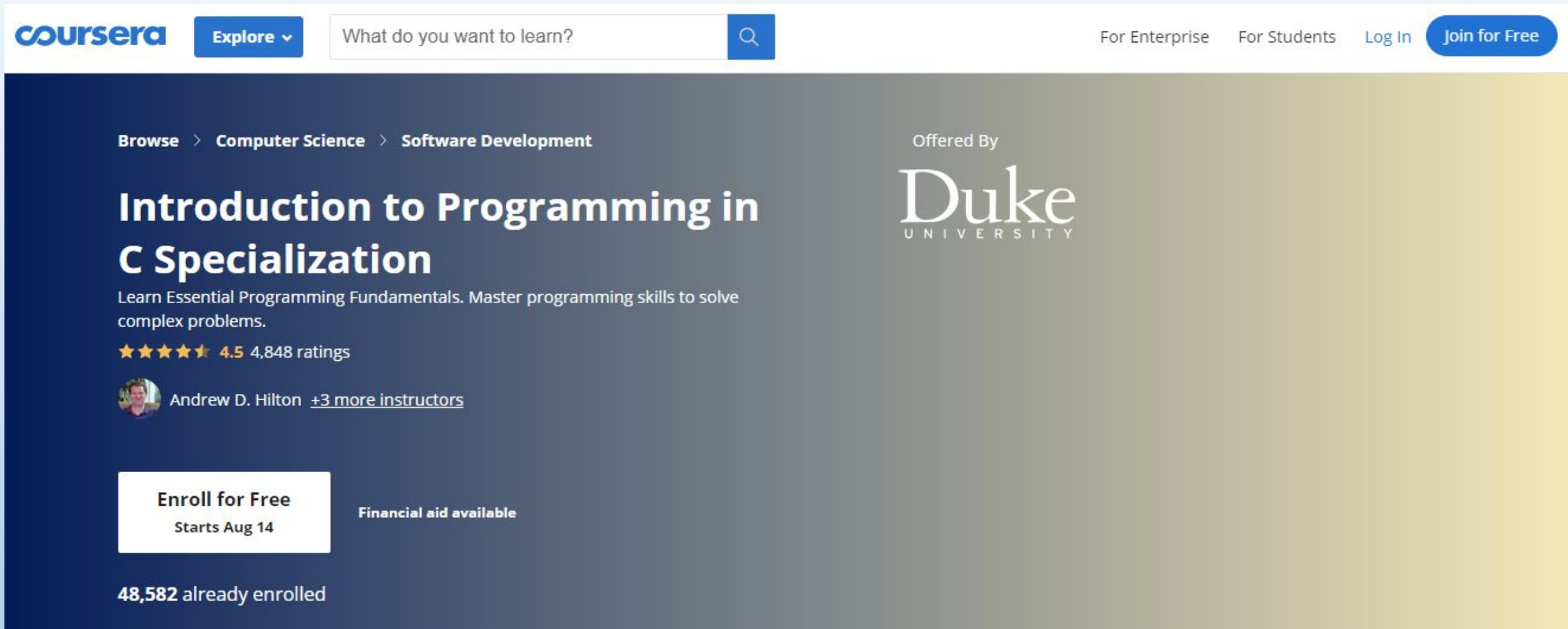
EnglishTamil

Show 10 entriesSearch:

Sl.No	Chapter Name	English
1	Intro - Process of programming	Download Verified

<https://nptel.ac.in/courses/106/104/106104128/#>
Introduction to Programming in C at NPTEL coordinated by IIT Kanpur

e-Resources (Courses (Theory/Practical) on Computer Programming using C)



The screenshot shows the Coursera website interface. At the top, the Coursera logo is on the left, followed by an 'Explore' button with a dropdown arrow. A search bar contains the text 'What do you want to learn?' with a magnifying glass icon. On the right, there are links for 'For Enterprise', 'For Students', 'Log In', and a 'Join for Free' button. Below the navigation bar, the breadcrumb trail reads 'Browse > Computer Science > Software Development'. The main title 'Introduction to Programming in C Specialization' is prominently displayed in white on a dark blue background. To the right, it says 'Offered By' followed by the Duke University logo. Below the title, a description states: 'Learn Essential Programming Fundamentals. Master programming skills to solve complex problems.' The course has a rating of 4.5 stars from 4,848 reviews. The instructor is listed as Andrew D. Hilton, with a link to '+3 more instructors'. A white button labeled 'Enroll for Free' is shown, with 'Starts Aug 14' below it. To the right of the button, it says 'Financial aid available'. At the bottom left, it indicates '48,582 already enrolled'.

Coursera Explore ▾ What do you want to learn? 🔍 For Enterprise For Students Log In Join for Free

Browse > Computer Science > Software Development

Introduction to Programming in C Specialization

Learn Essential Programming Fundamentals. Master programming skills to solve complex problems.

★★★★★ 4.5 4,848 ratings

 Andrew D. Hilton [+3 more instructors](#)

Enroll for Free
Starts Aug 14

Financial aid available

48,582 already enrolled

<https://www.coursera.org/specializations/c-programming>

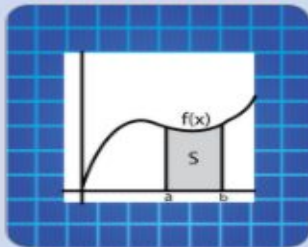
Introduction to Programming in C Specialization at Coursera offered by Duke University

e-Resources (Courses (Theory/Practical) on Computer Programming using C)

Computer Programming Lab

[Home](#)[About this lab](#)[Explore Vlabs](#)

1. Numerical Approximation



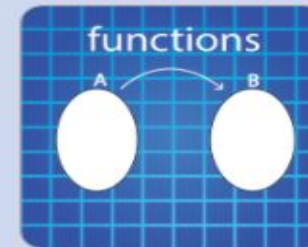
This experiment demonstrates the idea that one can attain a perfect solution in the limit by successively computing approximations to the exact value. The intent behind this experiment is to make use of ideas learned in the previous experiments. This is intended to introduce the student to numerical methods.

Objectives of this experiment are :

1. To understand that for some problems an approximate numerical solution is as good as the exact solution for practical purposes.

Prerequisite :

2. Functions



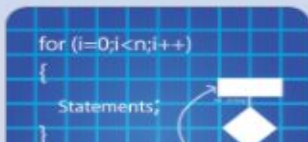
Functions enable modular decomposition of large programs. They allow reuse and improve readability and maintainability of programs. When constructed properly, functions also enable reasoning about their properties. A function typically receives arguments and returns a value. This experiment shows function definition and invocation within a program.

Objectives of this experiment are :

1. To understand that a big program can be broken up into independent modules.
2. To learn to define functions and call them with appropriate parameters.

Prerequisite :

3. Advanced Control Flow



This experiment shows the execution of programs that use for loops and while loops. It includes programs with nested for loops to illustrate complex control flow within the program. The student will be able to visually comprehend the flow control.

4. Arrays



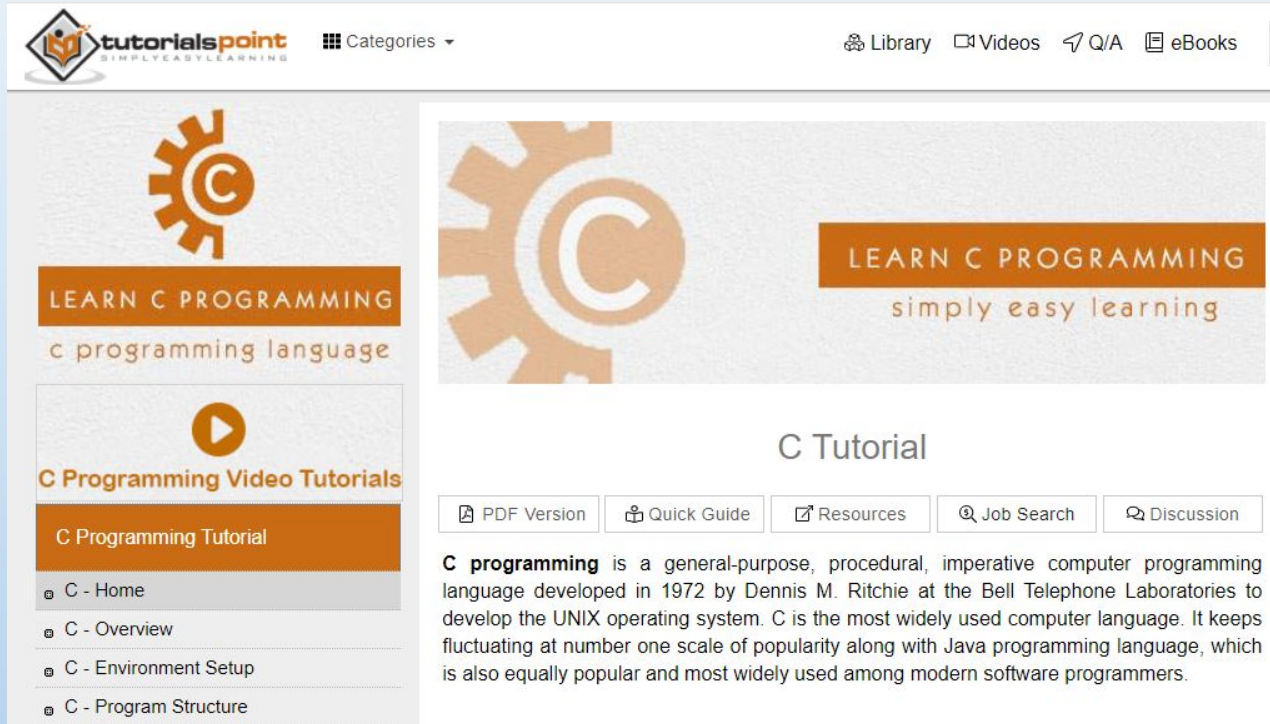
Arrays are used to store multiple data of same type. There are the most efficient means to store sequential collection of values. Programs use loops to process data stored in arrays. This experiment demonstrates steps involved in defining arrays, initializing them and processing the data contained in the arrays.

[http://cse02-iiith.vlabs.ac.](http://cse02-iiith.vlabs.ac.in/)

[in/](http://cse02-iiith.vlabs.ac.in/)

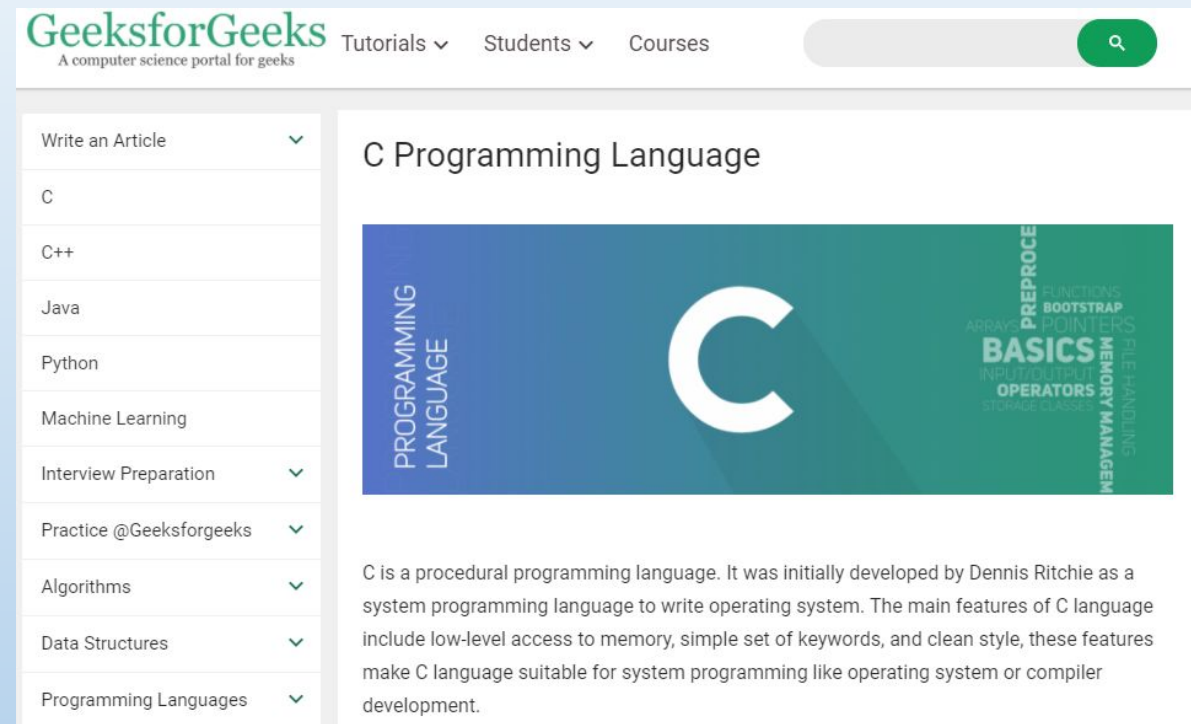
Virtual Lab: Computer Programming Lab created by IIIT Hyderabad

SDF – I: e-Resources (Tutorials/Coding Problems on Computer Programming using C)



The screenshot shows the 'tutorialspoint' website with the tagline 'SIMPLY EASY LEARNING'. The navigation bar includes 'Categories', 'Library', 'Videos', 'Q/A', and 'eBooks'. The main content area features a large orange gear logo with a 'C' inside, followed by the text 'LEARN C PROGRAMMING' and 'c programming language'. Below this is a 'C Programming Video Tutorials' section with a play button icon. A sidebar on the left lists 'C - Home', 'C - Overview', 'C - Environment Setup', and 'C - Program Structure'. The main text area is titled 'C Tutorial' and includes links for 'PDF Version', 'Quick Guide', 'Resources', 'Job Search', and 'Discussion'. The introductory paragraph states: 'C programming is a general-purpose, procedural, imperative computer programming language developed in 1972 by Dennis M. Ritchie at the Bell Telephone Laboratories to develop the UNIX operating system. C is the most widely used computer language. It keeps fluctuating at number one scale of popularity along with Java programming language, which is also equally popular and most widely used among modern software programmers.'

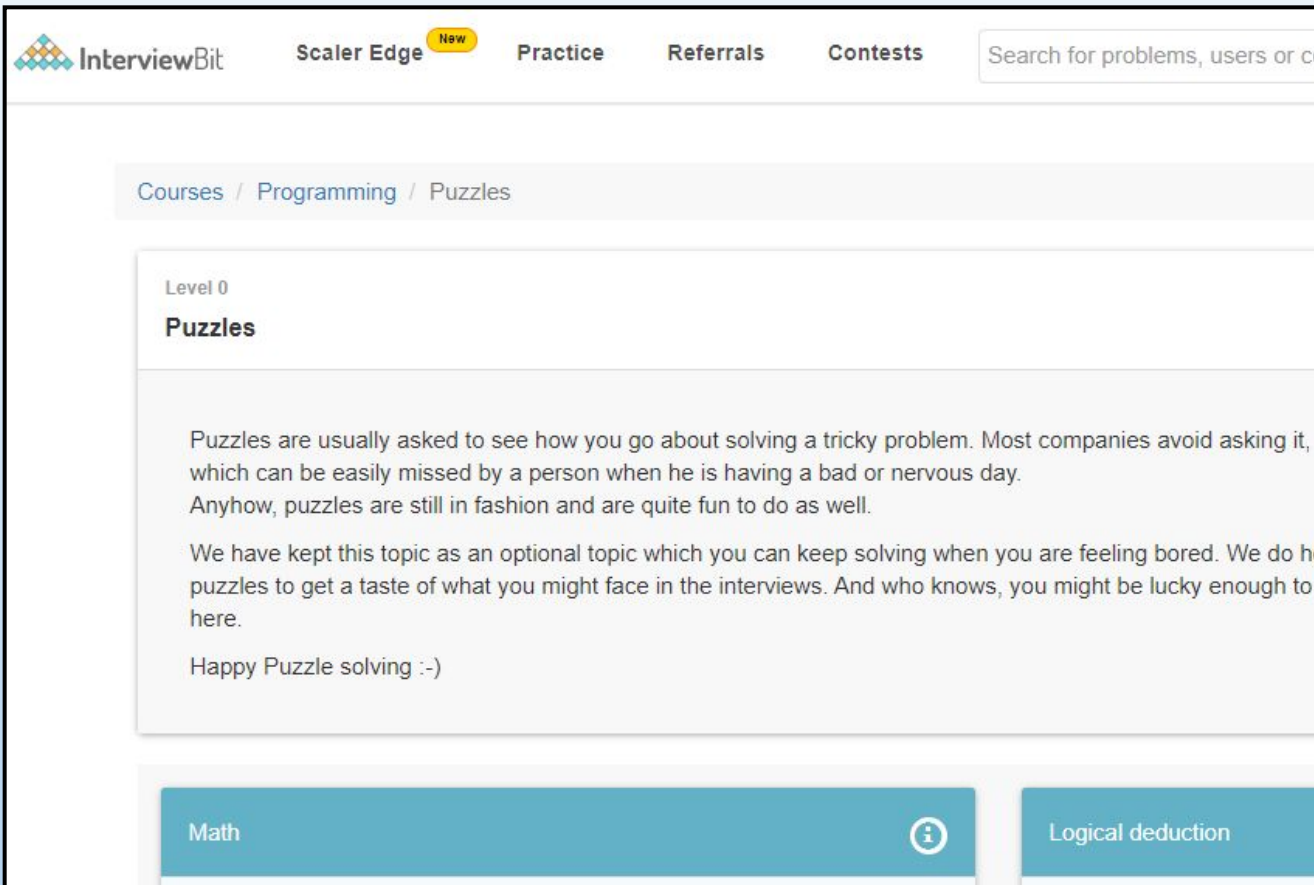
<https://www.tutorialspoint.com/cprogramming/index.htm>



The screenshot shows the 'GeeksforGeeks' website with the tagline 'A computer science portal for geeks'. The navigation bar includes 'Tutorials', 'Students', and 'Courses'. The main content area is titled 'C Programming Language' and features a large blue and green banner with a 'C' logo and the text 'PROGRAMMING LANGUAGE' and 'BASICS'. Below the banner, the text states: 'C is a procedural programming language. It was initially developed by Dennis Ritchie as a system programming language to write operating system. The main features of C language include low-level access to memory, simple set of keywords, and clean style, these features make C language suitable for system programming like operating system or compiler development.'

<https://www.geeksforgeeks.org/c-programming-language/>

e-Resources (Problems/Exercises on Puzzles)



The screenshot shows the InterviewBit website interface. At the top, there is a navigation bar with links for InterviewBit, Scaler Edge (marked as 'New'), Practice, Referrals, and Contests. A search bar is also present. Below the navigation bar, a breadcrumb trail indicates the path: Courses / Programming / Puzzles. The main content area is titled 'Level 0 Puzzles'. It contains a paragraph explaining that puzzles are often used in interviews to test problem-solving skills and are fun to solve. It also mentions that the site has kept puzzles as an optional topic for users to practice. At the bottom, there are two buttons: 'Math' and 'Logical deduction'.

InterviewBit Scaler Edge ^{New} Practice Referrals Contests Search for problems, users or c

Courses / Programming / Puzzles

Level 0
Puzzles

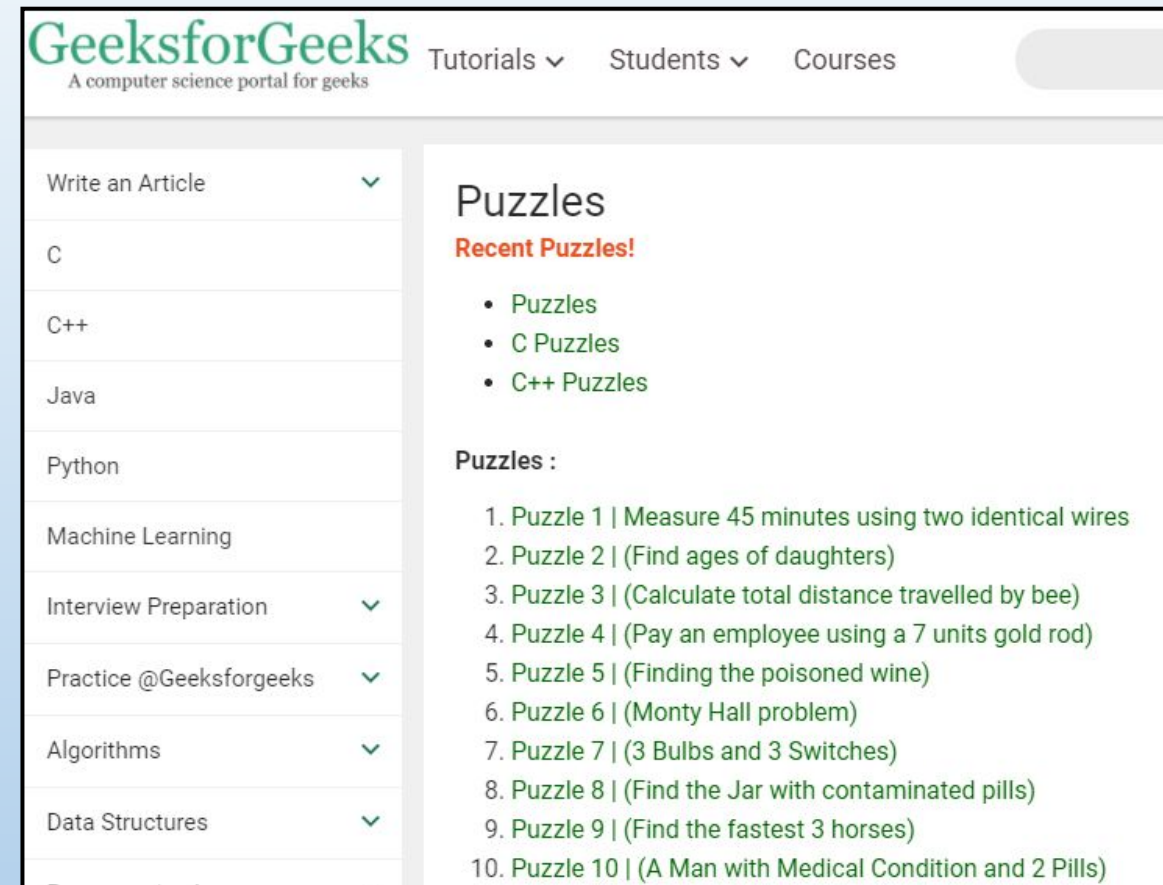
Puzzles are usually asked to see how you go about solving a tricky problem. Most companies avoid asking it, which can be easily missed by a person when he is having a bad or nervous day. Anyhow, puzzles are still in fashion and are quite fun to do as well.

We have kept this topic as an optional topic which you can keep solving when you are feeling bored. We do h puzzles to get a taste of what you might face in the interviews. And who knows, you might be lucky enough to here.

Happy Puzzle solving :-)

Math Logical deduction

<https://www.interviewbit.com/puzzles/#problems>



The screenshot shows the GeeksforGeeks website interface. At the top, there is a navigation bar with links for GeeksforGeeks, Tutorials, Students, and Courses. A search bar is also present. Below the navigation bar, a sidebar on the left lists various topics: Write an Article, C, C++, Java, Python, Machine Learning, Interview Preparation, Practice @Geeksforgeeks, Algorithms, and Data Structures. The main content area is titled 'Puzzles' and features a section for 'Recent Puzzles!'. It lists 10 puzzles, each with a brief description of the problem.

GeeksforGeeks A computer science portal for geeks Tutorials Students Courses

Write an Article
C
C++
Java
Python
Machine Learning
Interview Preparation
Practice @Geeksforgeeks
Algorithms
Data Structures

Puzzles
Recent Puzzles!

- Puzzles
- C Puzzles
- C++ Puzzles

Puzzles :

- Puzzle 1 | Measure 45 minutes using two identical wires
- Puzzle 2 | (Find ages of daughters)
- Puzzle 3 | (Calculate total distance travelled by bee)
- Puzzle 4 | (Pay an employee using a 7 units gold rod)
- Puzzle 5 | (Finding the poisoned wine)
- Puzzle 6 | (Monty Hall problem)
- Puzzle 7 | (3 Bulbs and 3 Switches)
- Puzzle 8 | (Find the Jar with contaminated pills)
- Puzzle 9 | (Find the fastest 3 horses)
- Puzzle 10 | (A Man with Medical Condition and 2 Pills)

<https://www.geeksforgeeks.org/puzzles/>

Programming Hubs at IIIT

KNUTH - IIIT Programming Hub (Sector 62)
(<https://www.facebook.com/groups/jiit.knuth>)

Rapid Programming Hub (Sector 128)
(<https://www.facebook.com/groups/rph128>)

Open Source Developers Community
(<https://www.facebook.com/groups/jiitlug>)



**End of
Introduction to Course**

Software Development Fundamentals (SDF) – I ODD 2023



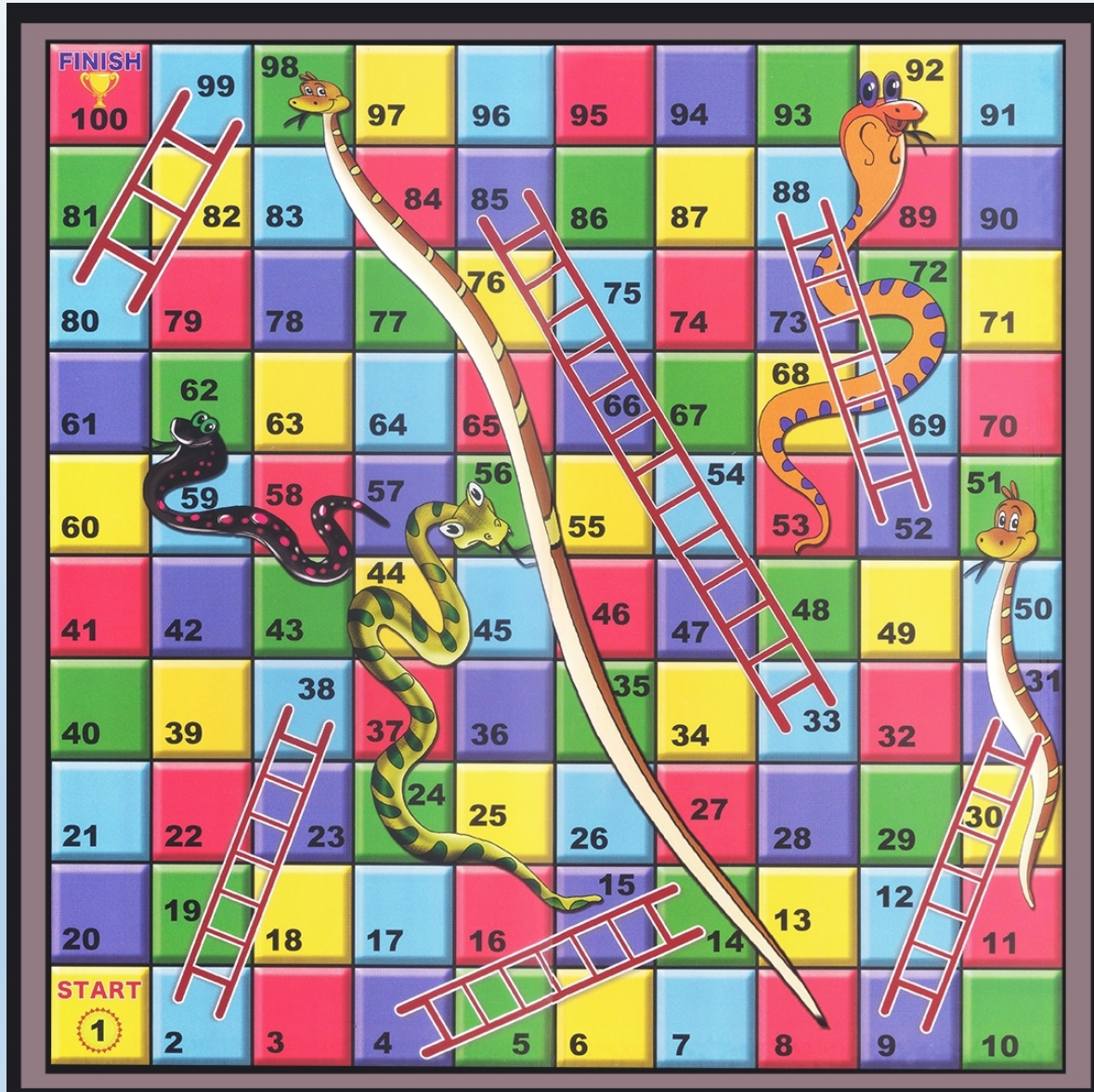
Lecture 1

Jaypee Institute of Information Technology (JIIT), Noida

Outline of Lecture

- **Snake & Ladder**
- **Tic-Tac-Toe**
- **Prime Number Verification**
- **Consolidate the Learnt Fundamentals**
- **Software Development Life Cycle**
- **Exercises**

Snake & Ladder (S&L)



Game Description (given by User)

1. $N \times N$ square grid, most popular is 10×10
2. Snakes and Ladders at different grid positions
3. Roll the dice and move the token forward
4. If token is at Head of Snake, move backward (Tail of Snake)
5. If token is at Bottom of Ladder, move forward (Top of Ladder)
6. Winning Condition: Token to be reached at N^2 grid position
i.e. 100, if it is 10×10 square grid

Q1: Did we forget any other point to describe the game?

Ans: No, the description covered everything

Q2: How goal position will be achieved?

Ans: No mechanism, totally depends on dice

Q3: What does Game Description represent?

Ans: It is user's **REQUIREMENTS** for S&L game

Source:

<http://www.nepaliclass.com/wp-content/uploads/2018/04/snake-and-ladder-board-game.jpg>

Snake & Ladder (S&L)

Developer's requirements to Design/Implement and Develop the S&L Game



- **REQUIREMENTS** (identify the requirements for system to be developed)

1. $N \times N$ square grid where N = customized/inputted by the user
2. Placing Head & Tail of Snakes and Top & Bottom of Ladders at different grid positions
3. A six sided dice (numbered between 1 and 6) and its rolling mechanism
4. Move the token as per the number obtained after rolling the dice
5. Forced backward movement, if token is placed at the Head of Snake
6. Forced forward movement, if token is placed at the Bottom of Ladder
7. Programming/coding language/platform/tools to implement/develop

Source:

<http://www.nepaliclass.com/wp-content/uploads/2018/04/snake-and-ladder-board-game.jpg>

Based on the identified **REQUIREMENTS** by the Developer (or Programmer) the System (S&L game) is to be:

- **DESIGNED**: Based on the requirements, complete architecture of the system including step by step solution need to be designed

e.g. where to place Head of the Snake so that there is possibility to win? Should we place it at each grid from 91 to 99

e.g. how rolling mechanism of the dice will work?

e.g. **Rules for playing the game** and **FLOW OF EVENTS** in the game:

- (1) Player 1 to roll the dice,
- (2) Based on dice value, move ahead and reach at Grid X
- (3) May have forced movement at X due to snake OR ladder
- (4) If X is destination grid, then Player 1 wins
- (5) Repeat 1 to 4 for Player 2

- **IMPLEMENTED**: Coding of the Designed System

Tic-Tac-Toe

(TTT)

Game Description (given by User)

1. $N \times N$ square grid, most popular is 3×3 . Centre grid (1 No), Corner grid (4 Nos.), and Side grid (4 Nos.)
2. Its two player (Player 1 and Player 2) game. Either Win/Loose or Draw
3. Tokens to be placed in each move are X (by Player 1) and 0 (by Player 2)
4. Winning Condition: If any of the grid position in Figure 1 is achieved by either of the player (Player 1 with X and Player 2 with 0)

Q1: Did we forget any other point to describe the game?

Ans: No, the game description given by the user is complete

Q2: Can we develop the TTT game with these descriptions?

Ans: No, besides above **REQUIREMENTS** it is also needed to **DESIGN** the **LOGIC** to achieve one of the winning condition

Q3: From Developer's perspective, any difference between S&L and TTT

Ans: Yes, besides development requirements, in TTT it is also needed to **DESIGN** the **LOGIC** to achieve the winning condition

Let's **DESIGN** the **LOGIC** (by playing the game of TTT)

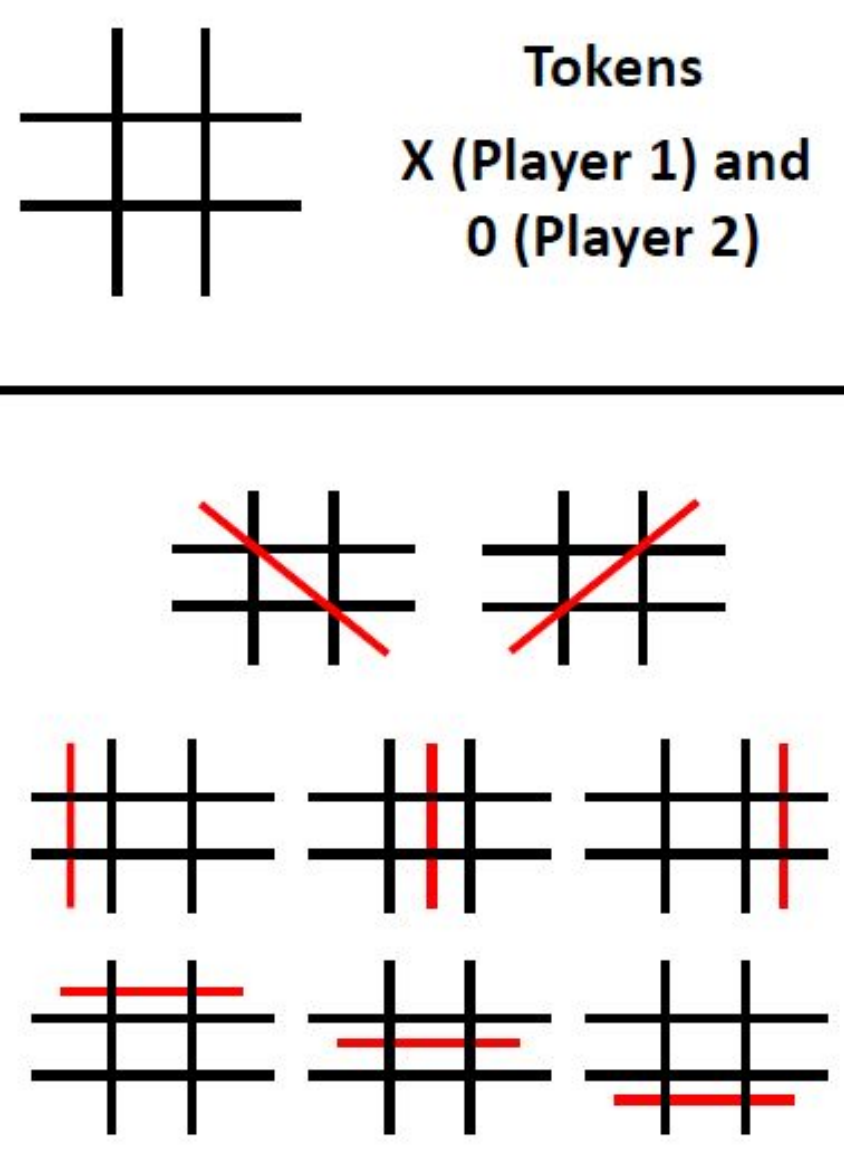
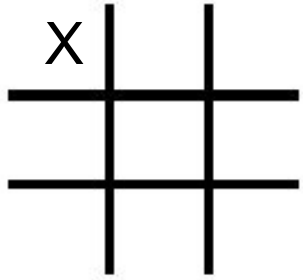


Figure 1: Winning Conditions (total 8)

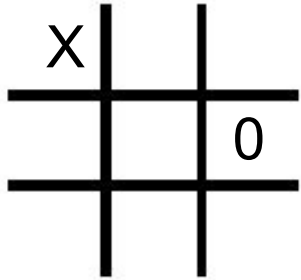
Tic-Tac-Toe

(TTT)

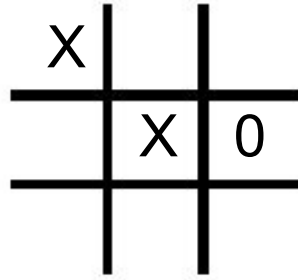
Lets play with Design Strategy 1 (DS1): No Strategy/Logic, randomly place X or 0



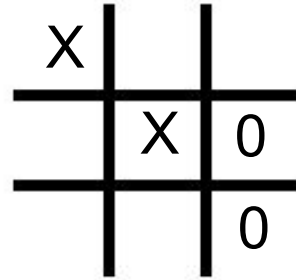
Player 1



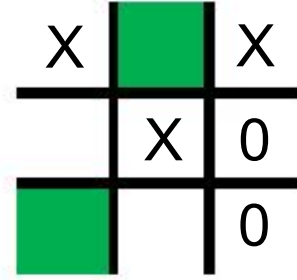
Player 2 (Computer)



Player 1



Player 2 (Computer)

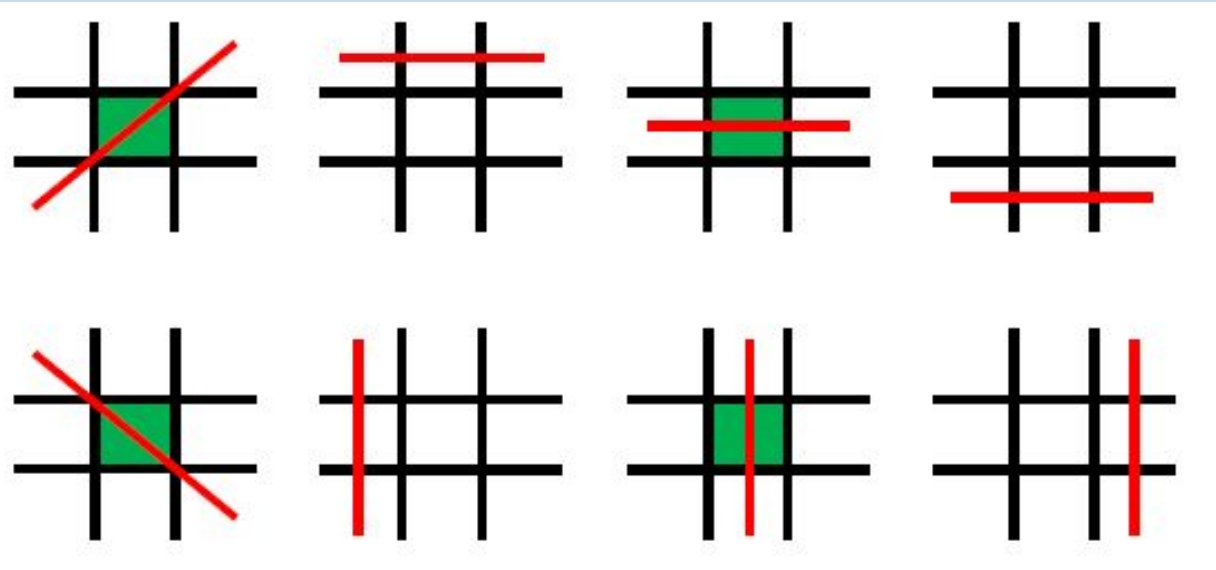


Player 1

Inference: Often players will win/lose the game

(Player 2 (Computer) will LOOSE)

Lets **DESIGN better** (refine the strategy) □ HOW ????



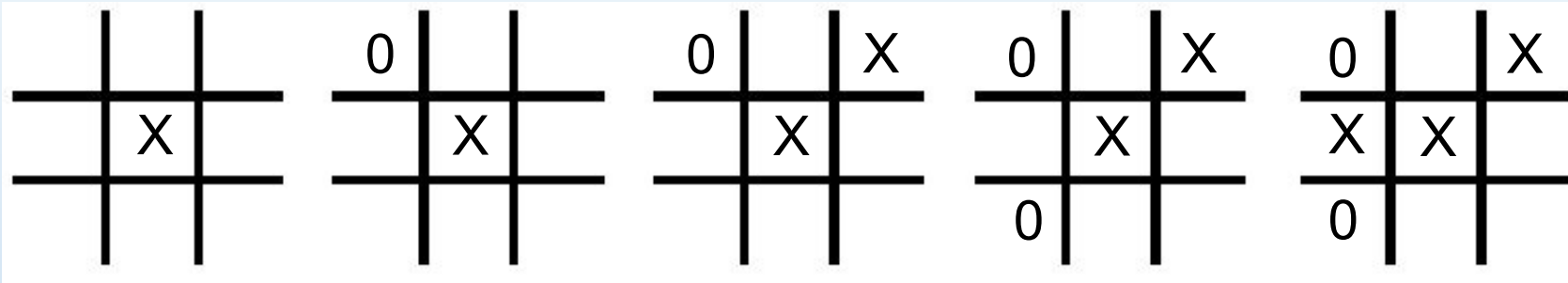
Argument: Out of 8, the centre grid is included in 4 winning conditions

i.e. to win or not to loose, **prime focus should be to occupy the centre grid** and increase the winning probability or narrow down the losing probability

Tic-Tac-Toe

(TTT)

Lets play with Design Strategy 2 (DS2): Don't leave the central grid empty



Player 1

Player 2 (Computer)

Player 1

Player 2 (Computer)

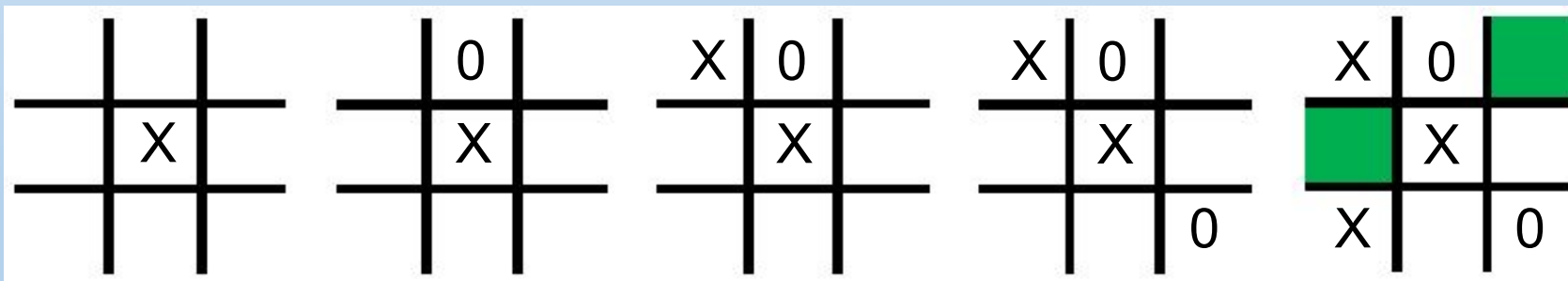
Player 1

Game will DRAW (Players will not loose)

Q1: Will player **never** win/loose with DS2?

Ans: Let's Check / **TEST** with another example

Will player never loose with DS2? Lets Check / TEST



Player 1

Player 2 (Computer)

Player 1

Player 2 (Computer)

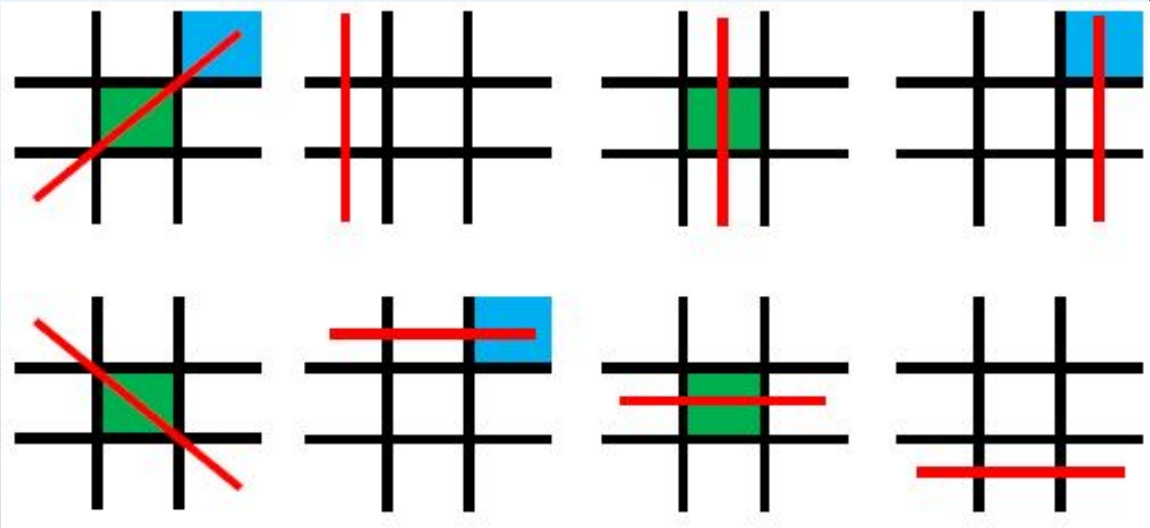
Player 1

(Player 2 (Computer) will LOOSE)

Inference: Players **may** win/loose with DS2

Lets **DESIGN better** (further refine the strategy) □ **HOW** ????

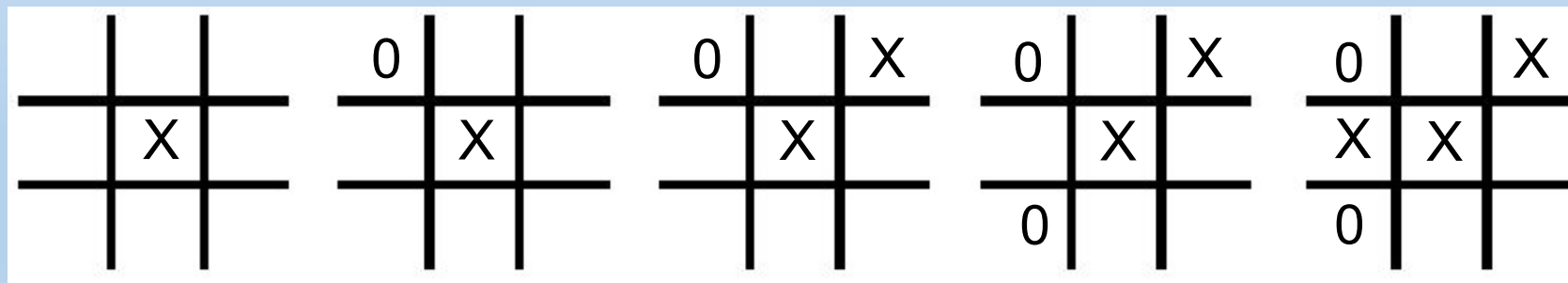
Tic-Tac-Toe (TTT)



Argument: Out of 8, the **centre grid** is included in 4 winning conditions & any **one of the corner grid** is included in 3 winning conditions

i.e. to win or not to loose, **prime focus** should be to occupy the **centre grid** and then on **corner grid**.

Lets play with Design Strategy 3 (DS3): DS2, then prefer to fill the corner grids first



Player 1

Player 2 (Computer)

Player 1

Player 2 (Computer)

Player 1

Game will DRAW

Q1: Will player never win/loose with DS3?

Ans: Think Yourself (*i.e.* **TEST** with another Example)

Suggestion: If needed, further refine the **DESIGN**

Prime Number Verification (PNV)

Given: An integer, N

Objective: Need to verify, N is prime number or not

Strategy / Logic : 1 (S1)	Strategy / Logic : 2 (S2)	Strategy / Logic : 3 (S3)
REQUIREMENTS: Need to input N DESIGN LOGIC: If N is divisible by M, where $M \geq 2$ and $M < N$ then N is not a Prime Number TEST: Example 1: Say N = 25 M between 2 & 24 Divide 25 by 2, 3, 4, 5 (it is divisible) 25 is divisible by M, so, Not-Prime Example 2: Say N = 11 M between 2 & 10 Divide 11 by 2, 3, 4, 5, 6, 7, 8, 9, 10 11 is not divisible by M, so, Prime Q1: Is S1 correct every time Ans: Yes. TEST it with more examples	REQUIREMENTS: Need to input N DESIGN LOGIC: If N is divisible by M, where $M \geq 2$ and $M \leq (N / 2)$ then N is not a Prime Number TEST: Example 1: Say N = 25 M between 2 & 13 (25 / 2) Divide 25 by 2, 3, 4, 5 (it is divisible) 25 is divisible by M, so, Not-Prime Example 2: Say N = 11 M between 2 & 6 (11 / 2) Divide 11 by 2, 3, 4, 5, 6 11 is not divisible by M, so, Prime Q1: Is S2 correct every time Ans: Yes. TEST it with more examples	REQUIREMENTS: Need to input N DESIGN LOGIC: If N is divisible by M, where $M \geq 2$ and $M \leq (\sqrt{N})$ then N is not a Prime Number TEST: Example 1: Say N = 25 M between 2 & 5 ($\sqrt{25}$) Divide 25 by 2, 3, 4, 5 (it is divisible) 25 is divisible by M, so, Not-Prime Example 2: Say N = 11 M between 2 & 4 ($\sqrt{11}$) Divide 11 by 2, 3, 4 11 is not divisible by M, so, Prime Q1: Is S3 correct every time Ans: Yes. TEST it with more examples

Q1: Which strategy is better?

Hint: Are S1, S2, & S3 correct? Which strategy takes less time (*i.e.* time efficient)?

Ans: S3

Consolidate the Fundamentals learnt in S&L, TTT, & PNV

REQUIREMENTS: User's (or customer's) Requirement (refer the description given by users in S&L, TTT)

+

Developer's (or Programmer's) Requirement (refer the developer's requirement in S&L)

DESIGN: System Design (refer the game of S&L)

+

Design the Logic to accurately solve the problem (refer the DESIGN LOGIC in TTT)

+

Design the Logic to efficiently solve the problem (refer different strategies / Logic in PNV)

LOGIC to solve (often accurately and efficiently) the problem □ referred as **ALGORITHM**

ALGORITHM □ Step by step solution of a problem (*e.g.* TTT, PNV, *etc.*)

IMPLEMENTATION: Coding (in programming language, e.g. C/C++/Java) of the designed system/logic

TEST: Check the performance of the Designed System / Logic (refer TEST mechanism in TTT)

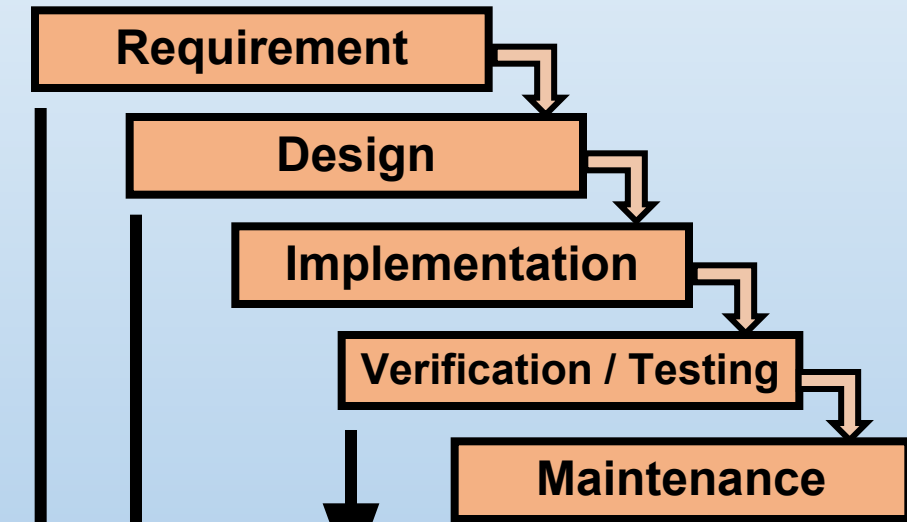
Need improvements (in terms of accuracy & efficiency), then refine/correct/improve the DESIGN

Software Development Life Cycle (SDLC)

SDLC represents all the steps/stages in development of a software product and its maintenance

Basic SDLC models: **Waterfall Model** and **Prototype Model**

Waterfall Model

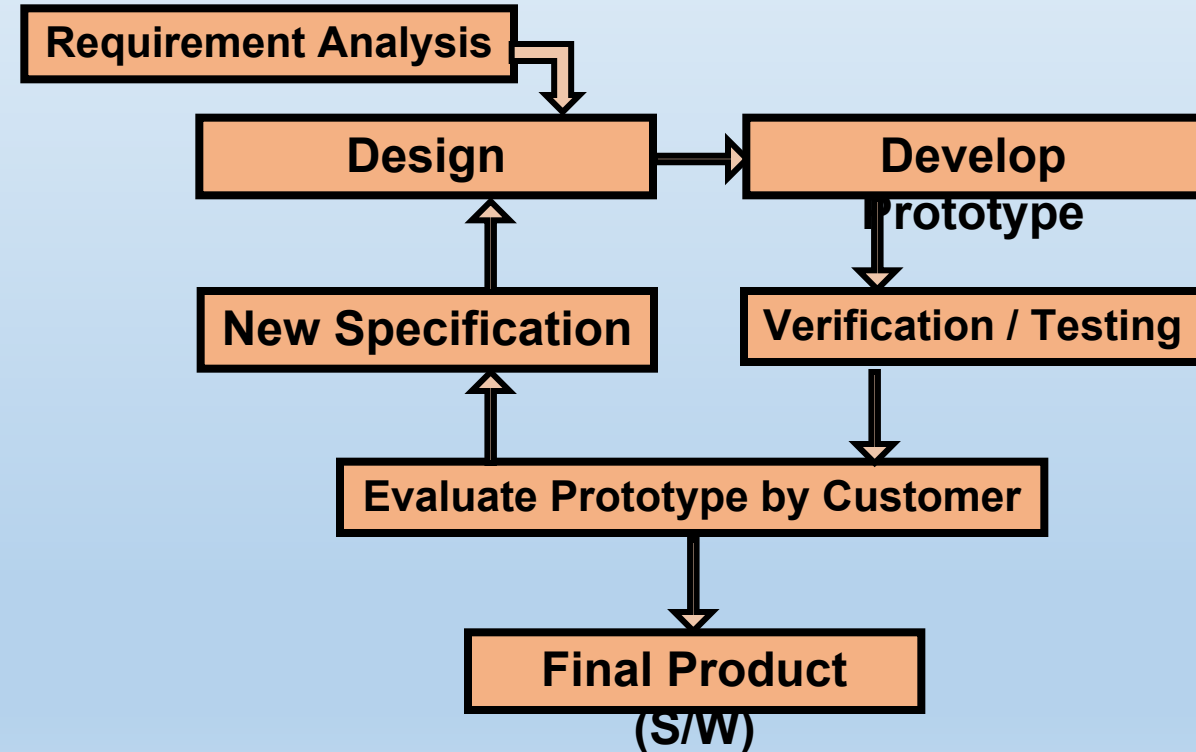


Testing of the developed system

Logic design and testing of the logic
Design of the complete system

Analysis of User's and Developer's requirements. Also, analyze the **feasibility, risk, timeline & incurred cost**

Prototype Model



Exercises

Analyze the Requirements and Flow of Events for following system:

Q1. Lift / Elevator control system in multi-story building □ **brief in Lecture 2 and discussion in Tutorial**

Q2. Intelligent Traffic control signal to control the traffic movements at squares □ **Discussion in Tutorial**

**End of
Lecture 1**